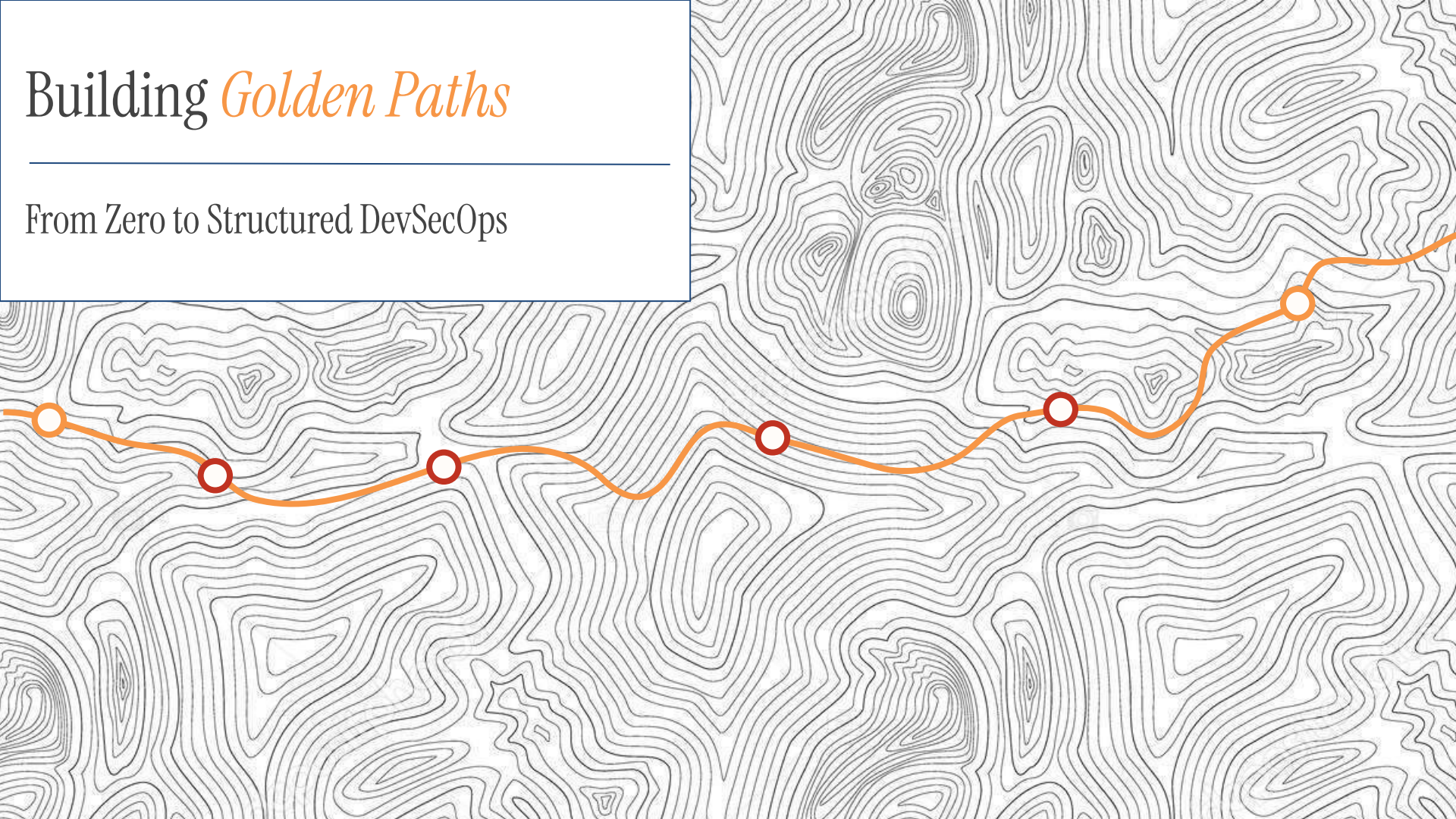
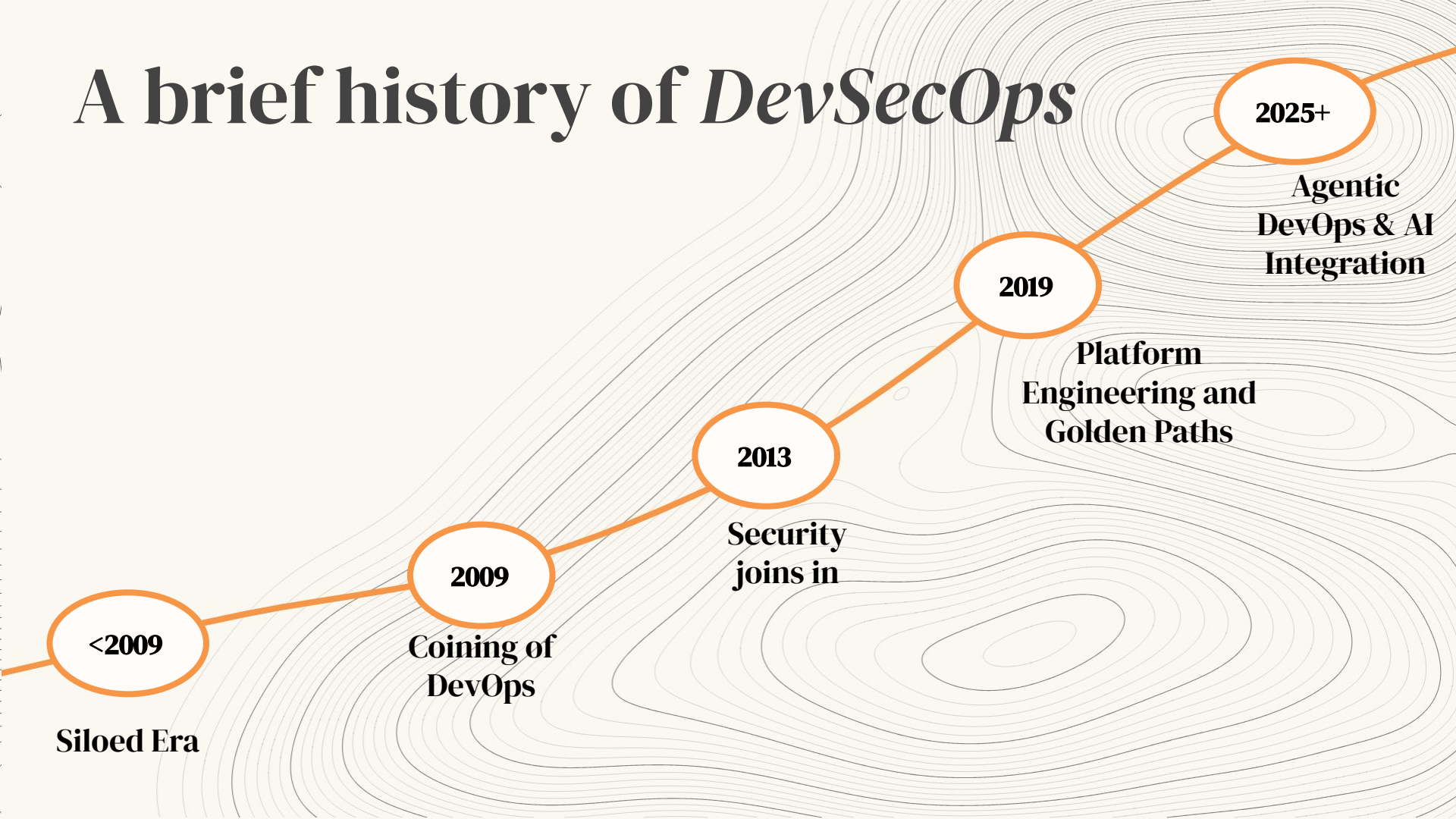
Building *Golden Paths*

From Zero to Structured DevSecOps



A brief history of *DevSecOps*

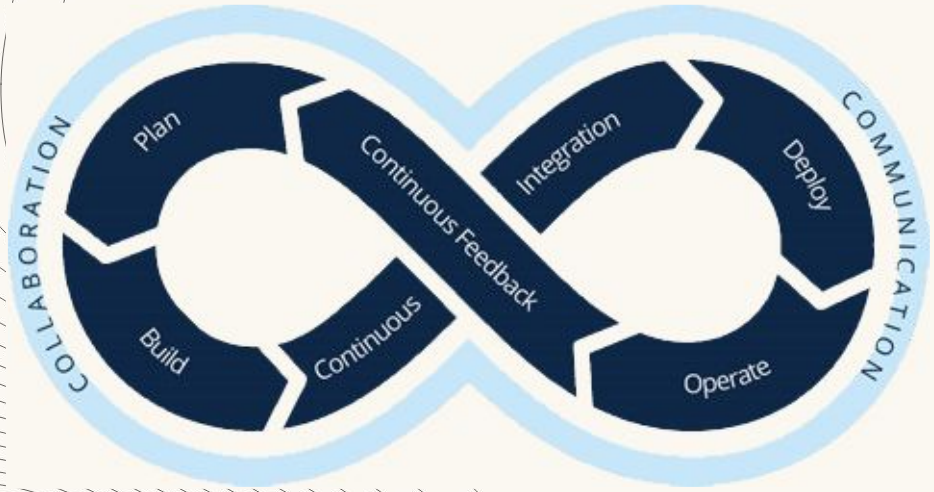


Siloed Era and Wall of Confusion

Developers wrote code and “threw it over the wall” for Operations to deploy and maintain

Developers were measured on velocity while Operations were measured on stability. These incentives were in conflict with one another

DevOps

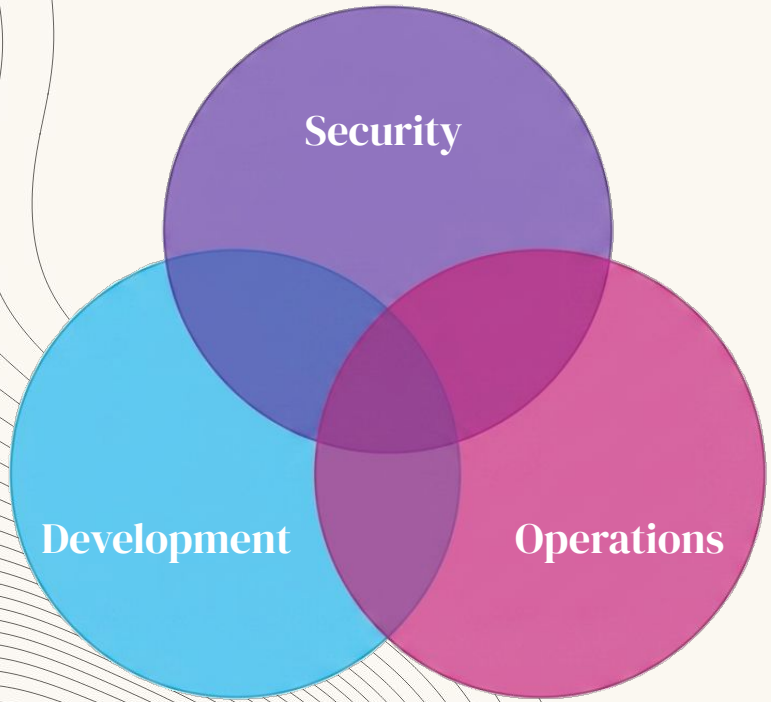


DevOps introduced a culture of shared responsibility and success.

Integrates Plan, Build, Deploy and Operate into the development lifecycle.

CI/CD Pipelines to automate the handoffs that created friction and to create a feedback loop

DevSecOps



Combines the cultural philosophies and practices of DevOps with Security to achieve frequent delivery of business value and automated enforcement of security.

Shifting checks earlier into development lifecycle to catch bugs when they are cheapest to fix

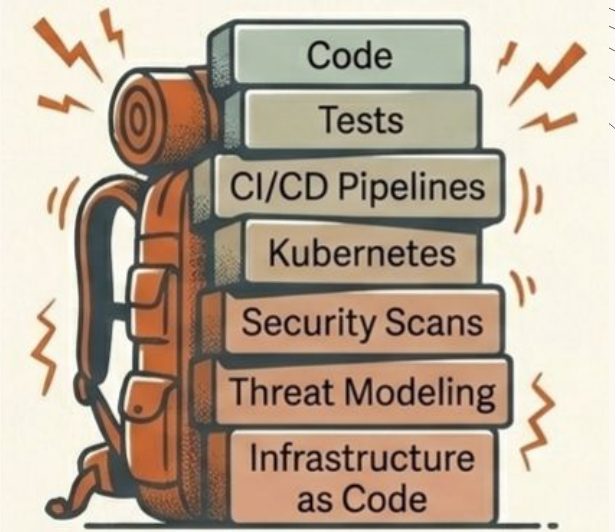
~~Shift Left~~ Throw Left

The well-intentioned mandate to Shift Left **aimed to move security earlier** in the lifecycle.

In practice, it simply **threw these huge responsibilities** onto the developer

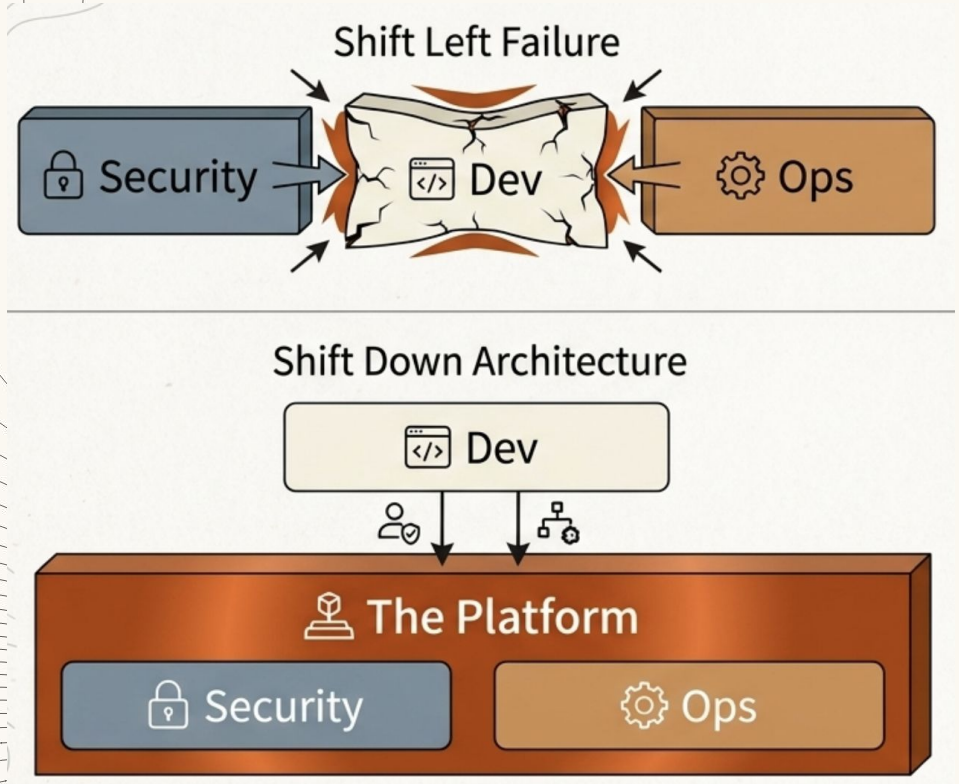
Cognitive Backpack

Traditional Developer



Modern Developer

The New Paradigm: Shift Down

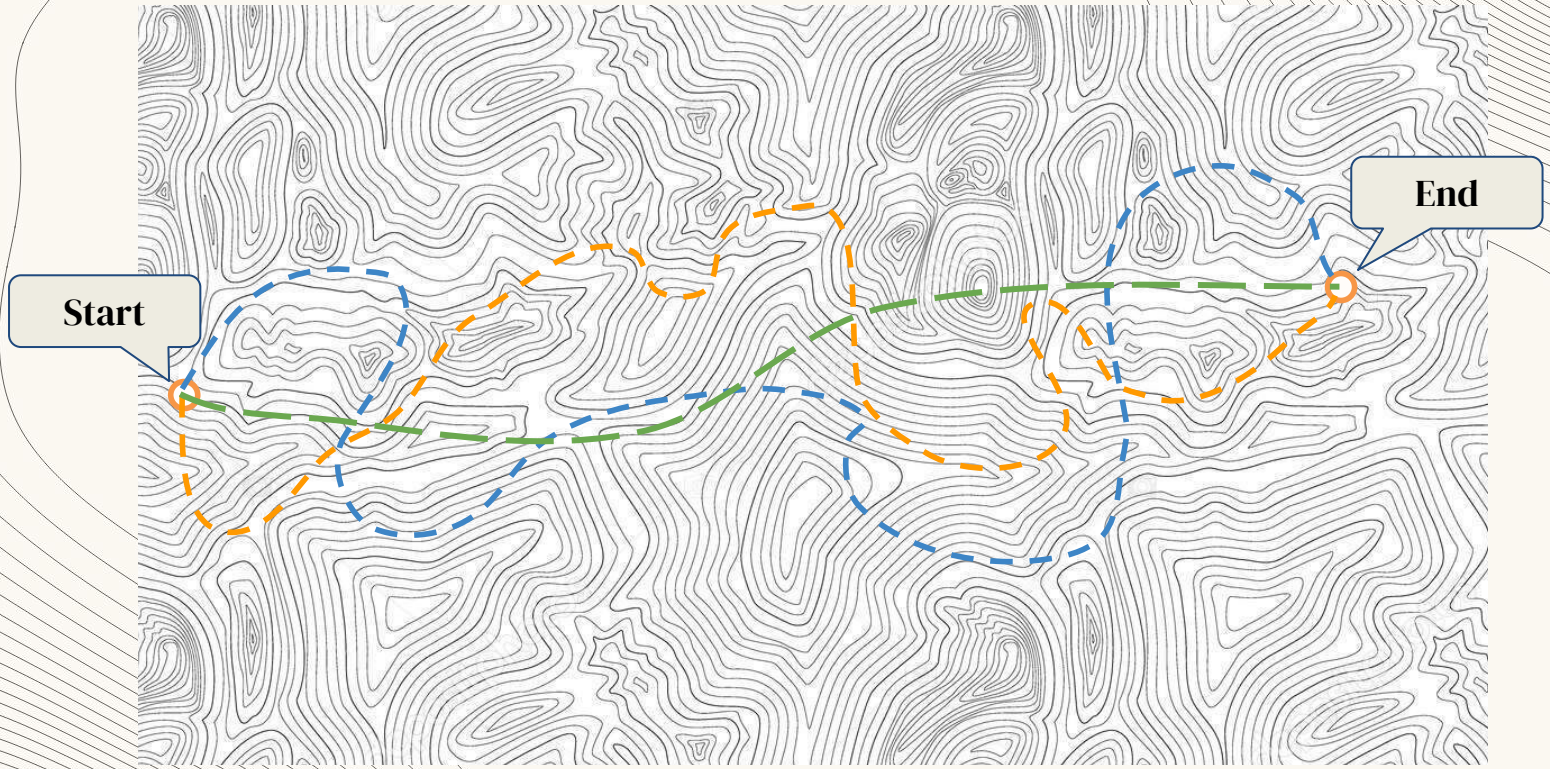


Shift down the underlying complexity of security and infrastructure to a specialised Platform Team.

This encapsulates deep domain knowledge, allowing developers to focus entirely on business logic.

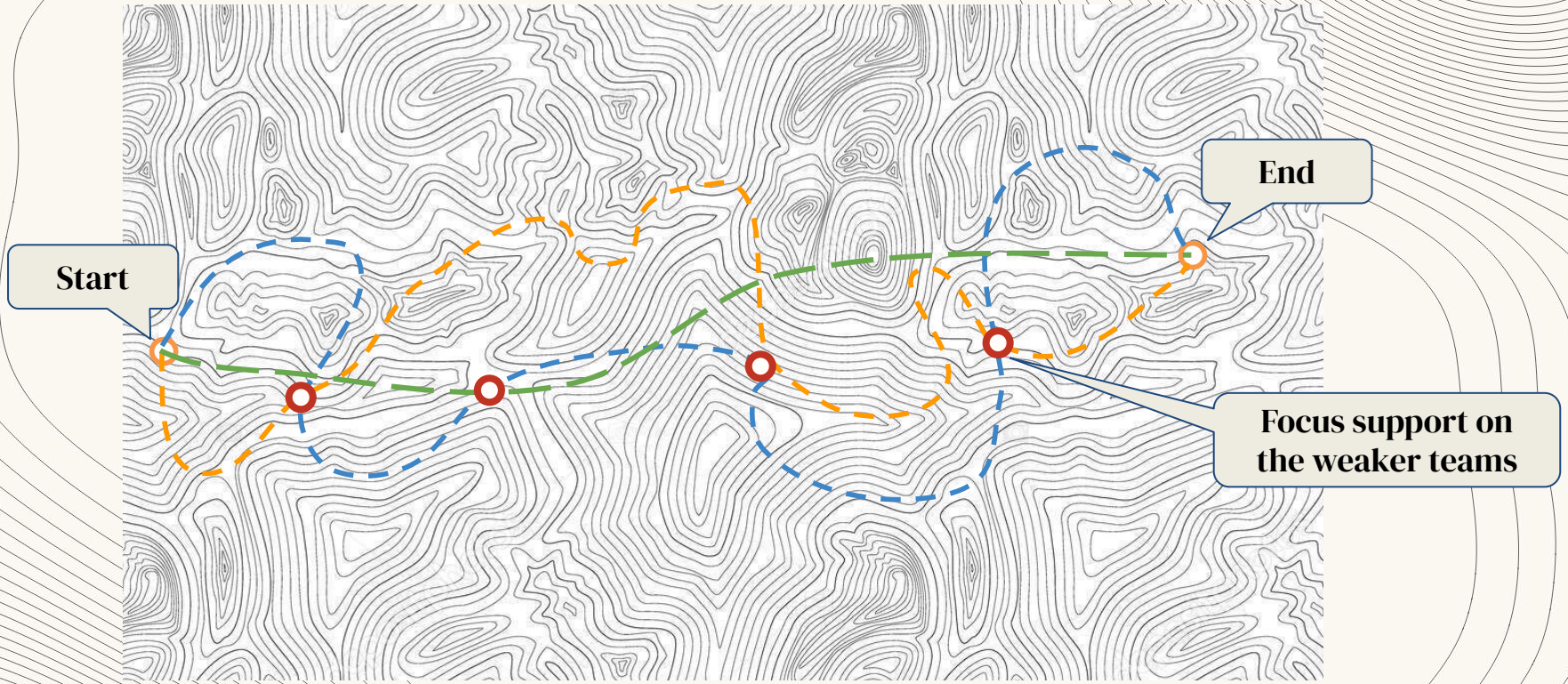
Security also becomes an intrinsic foundational property and not an external checklist

The Wild Wild West



Without clear guidance, every developer will embark on their own journey, on their routes

Trail Markers



Identified key points that we want developers to follow to actively avoid security pitfalls

Trail Markers

- Gave developers reusable jobs they can assemble themselves
- Teams extends the jobs that they need and wire together their own pipeline

Our first “Shift Down”: secret detection and code validation was included by default. Developers did not have to configure it. It just provides security

```yaml
secret_detection:          # 🔍 Scans for leaked secrets
  stage: .pre              #    Runs FIRST, before everything

tf-validate:               # ✅ "Is this valid Terraform?"
  extends: [.base-rules]
  stage: code-quality-and-security-checks
  script:
    - terraform init -backend=false
    - terraform validate

.tf-plan:                  # 📋 Reusable terraform plan job — teams extend this
  extends: [.invoke-awscli-commands-with-assumerole, .base-rules]
  image: aws-cli-and-terraform-container:latest
  script:
    - terraform init -backend-config=${BACKEND_FILE_PATH}
    - terraform plan -var-file=${VAR_FILE_PATH} -out=tfplan
  artifacts:
    paths: [tfplan]        #    Saves the plan for the apply step
```

Trail Markers

This is what each team's pipeline file looked like

```yaml
# The team's .gitlab-ci.yml — their pipeline config
include:                    # Pull in our shared templates
  - project: 'platform/ci-cd-templates'
    ref: main
    file: '/terraform/jobs.yml'

dev-plan:                   # Wire up planning for dev environment
  extends: .tf-plan         # ← Extend our template
  stage: dev
  variables:
    ROLE_ARN: "arn:aws:iam::123456789:role/dev-deploy"
    VAR_FILE_PATH: "env/dev.tfvars"
    BACKEND_FILE_PATH: "env/dev.backend.hcl"
```

Trail Markers

WHAT WORKED

- Standardised jobs consistent across teams
- Some security scanning and checks by default
- No more “writing from scratch” for every project
- Improvement in the template benefits all teams

WHAT HURT

- Teams still assembled manually leading to slight differences
- Needed DevOps expertise to wire it correctly
- Security controls only ran if teams assembled them correctly

Trail markers without a paved route leave the route opened to interpretation - a hiker can still get lost
Inconsistent security controls can be as dangerous as no security controls

Golden Path

An opinionated, secure-by-default route that accelerates delivery by making the right way the absolute path of least resistance

Golden Path

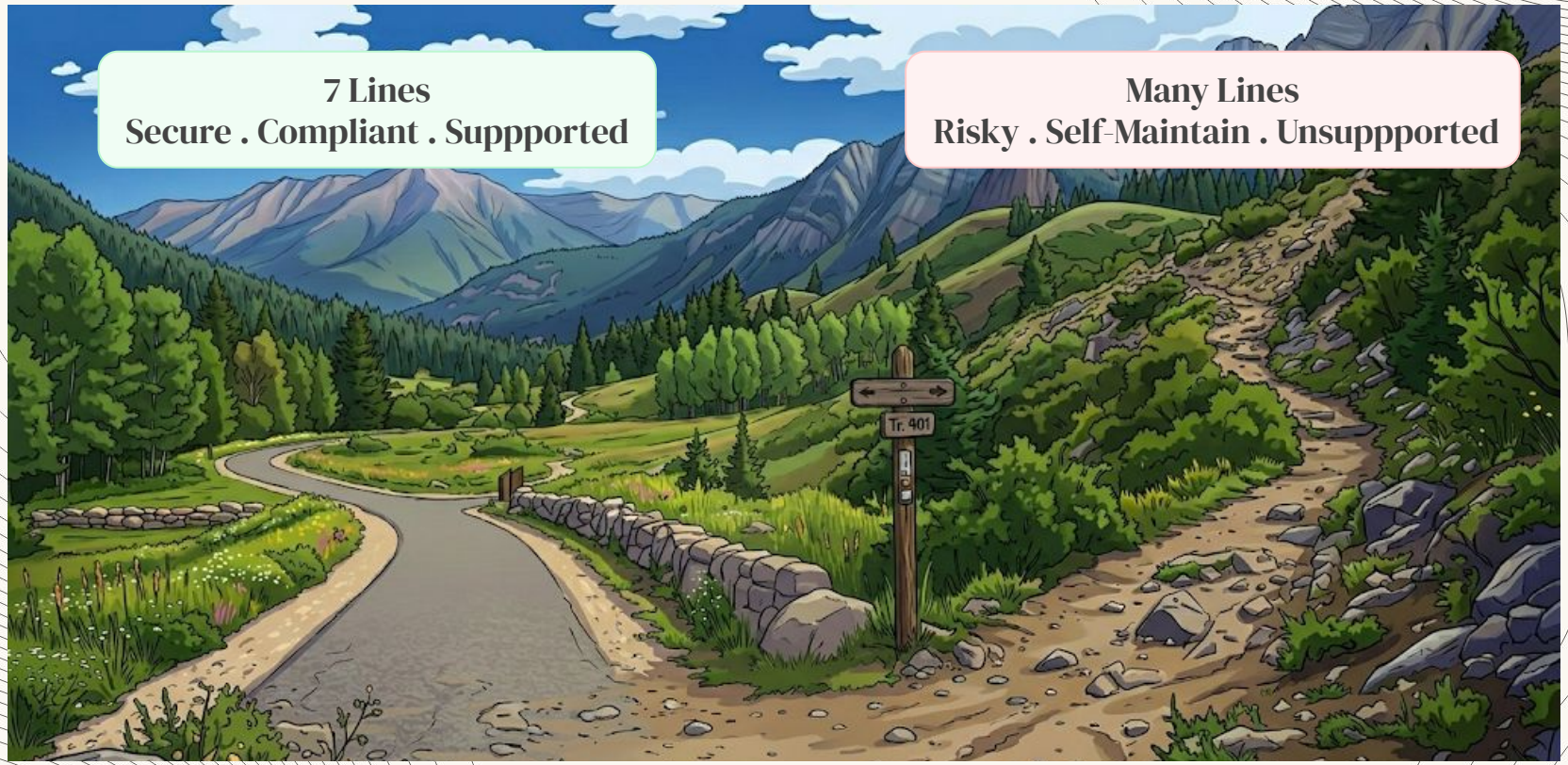
- From “here are some reusable jobs” to “here is THE way”
- Coined by Spotify to solve “rumour-driven development”. Netflix calls it “Paved Road”
- Make doing the right thing the easy thing to reduce cognitive load and increase velocity

```yaml
include:
  - component: gitlab.example.com/platform/ci-cd-templates/terraform@v2
    inputs:
      job_label: "payments-infra"
      dev_role_arn: "arn:aws:iam::111111111:role/dev-deploy"
      uat_role_arn: "arn:aws:iam::222222222:role/uat-deploy"
      prd_role_arn: "arn:aws:iam::333333333:role/prd-deploy"
```

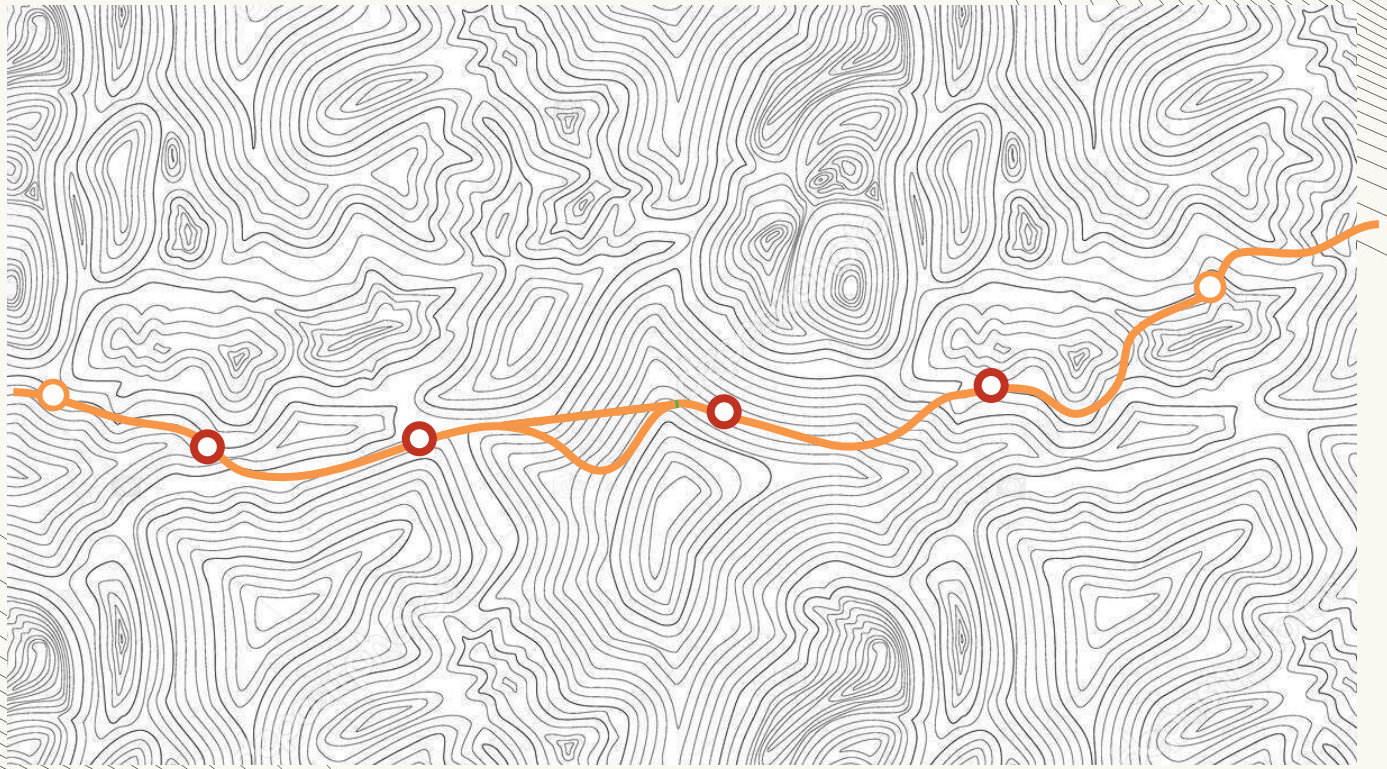
That’s it. Seven Lines.

What every team gets automatically without any further configuration:
Secret Detection. IaC security scanning. SAST scans. Code Validation. Multi-environment deployment. Standardised approval gates before production deployment

Golden Path


7 Lines
Secure . Compliant . Suppported
Many Lines
Risky . Self-Maintain . Unsuppported
Tr. 401

Off Roading



Golden Paths should not trap developers. Skilled adventurers should be free to explore new routes

Off Roading

OPINIONATED != LOCKED DOWN

- On the path: full support, automatic security, compliance handled
- Off-road: You own the maintenance and the security risks
- Yesterday's offroad becomes tomorrow's Golden Path

TREAT TEMPLATE AS PRODUCTS

- Templates have users, versions, changelogs, documentations
- Track adoption, time-to-first-deploy, override frequency
- When teams consistently override controls, that is user feedback
- Security controls should be a product developers consume

If teams are working around your security controls, that is a design problem, not a compliance problem

Secure your Destination by Guiding the Journey

1 Stop throwing left. Shift down to encapsulate complexity.

2 Reduce developer cognitive load to increase delivery velocity

3 Build Golden Paths that make the secure way the path of least resistance

Thank You